

Para atender ao seu cenário de cadastro e listagem de animais em uma clínica veterinária, estruturei a arquitetura seguindo os princípios da **Arquitetura Limpa (Clean Architecture)** e **SOLID**. O foco é manter as regras de negócio isoladas de ferramentas externas, garantindo que o núcleo da aplicação seja fácil de testar e manter 1-3. Abaixo está a estrutura de entidades, casos de uso e interfaces necessária para o seu cenário: **### 1. Camada de Domínio (Entidades)** As entidades representam os objetos de negócio e suas regras fundamentais 4, 5. Aqui, o animal de estimação é o centro do sistema. typescript // src/domain/entities/pet.ts import { randomUUID } from "node:crypto"; export interface PetProps { name: string; species: string; breed: string; age: number; } export class Pet { private _id: string; private props: PetProps; constructor(props: PetProps, id?: string) { this._id = id ?? randomUUID(); // Geração de ID no domínio para independência [6] this.props = props; } get id() { return this._id; } get name() { return this.props.name; } get species() { return this.props.species; } get breed() { return this.props.breed; } get age() { return this.props.age; } } **### 2. Camada de Aplicação (Casos de Uso e Interfaces)** Esta camada contém as ações que o sistema executa (casos de uso) e os contratos para comunicação com dados externos (repositórios) 7-9. **##### Contrato do Repositório (Interface)** O repositório define o **que** deve ser feito, sem expor **como** (banco de dados ou memória) 10, 11. typescript // src/application/repositories/IPetRepository.ts import { Pet } from "../domain/entities/pet"; export interface IPetRepository { create(pet: Pet): Promise; findAll(): Promise; } **##### Casos de Uso (SOLID: Princípio da Responsabilidade Única)** Cada classe abaixo possui apenas uma única tarefa no sistema 12, 13. typescript // src/application/use-cases/register-pet-use-case.ts import { Pet } from "../domain/entities/pet"; import { IPetRepository } from "../repositories/IPetRepository"; interface IRegisterPetRequestDTO { name: string; species: string; breed: string; age: number; } export class RegisterPetUseCase { // Inversão de dependência: depende da interface, não da implementação [14, 15] constructor(private petRepository: IPetRepository) {} async execute(request: IRegisterPetRequestDTO): Promise { const pet = new Pet(request); await this.petRepository.create(pet); } } // src/application/use-cases/list-pets-use-case.ts export class ListPetsUseCase { constructor(private petRepository: IPetRepository) {} async execute(): Promise { return await this.petRepository.findAll(); } } **### 3. Camada de Adaptadores (Controller)** O Controller recebe as requisições (como do Express) e as repassa para os Casos de Uso 16-18. typescript // src/adapters/controllers/pet-controller.ts import { RegisterPetUseCase } from "../application/use-cases/register-pet-use-case"; import { ListPetsUseCase } from "../application/use-cases/list-pets-use-case"; export class PetController { constructor(private registerPet: RegisterPetUseCase, private listPets: ListPetsUseCase) {} async handleRegister(req: any, res: any) { const { name, species, breed, age } = req.body; await this.registerPet.execute({ name, species, breed, age }); return res.status(201).send(); } async handleList(req: any, res: any) { const pets = await this.listPets.execute(); return res.status(200).json(pets); } } **### 4. Camada de Infraestrutura (Implementação)** Implementação concreta do repositório. Para o seu cenário inicial, um repositório em memória permite o funcionamento imediato sem a necessidade de configurar um banco de dados 19, 20. typescript // src/infrastructure/repositories/in-memory-pet-repository.ts import { IPetRepository } from "../application/repositories/IPetRepository"; import { Pet } from "../domain/entities/pet"; export class InMemoryPetRepository implements IPetRepository { public items: Pet[] = []; async create(pet: Pet): Promise { this.items.push(pet); } async findAll(): Promise { return this.items; } } Esta arquitetura garante que, se no futuro você decidir usar um banco de dados como PostgreSQL com Prisma, precisará apenas criar uma

nova implementação de IPetRepository sem alterar nenhuma regra de negócio ou controle já existente 21, 22.